

再谈octomap——一些细节

SLAM

📅 2021年 02月25日

背景

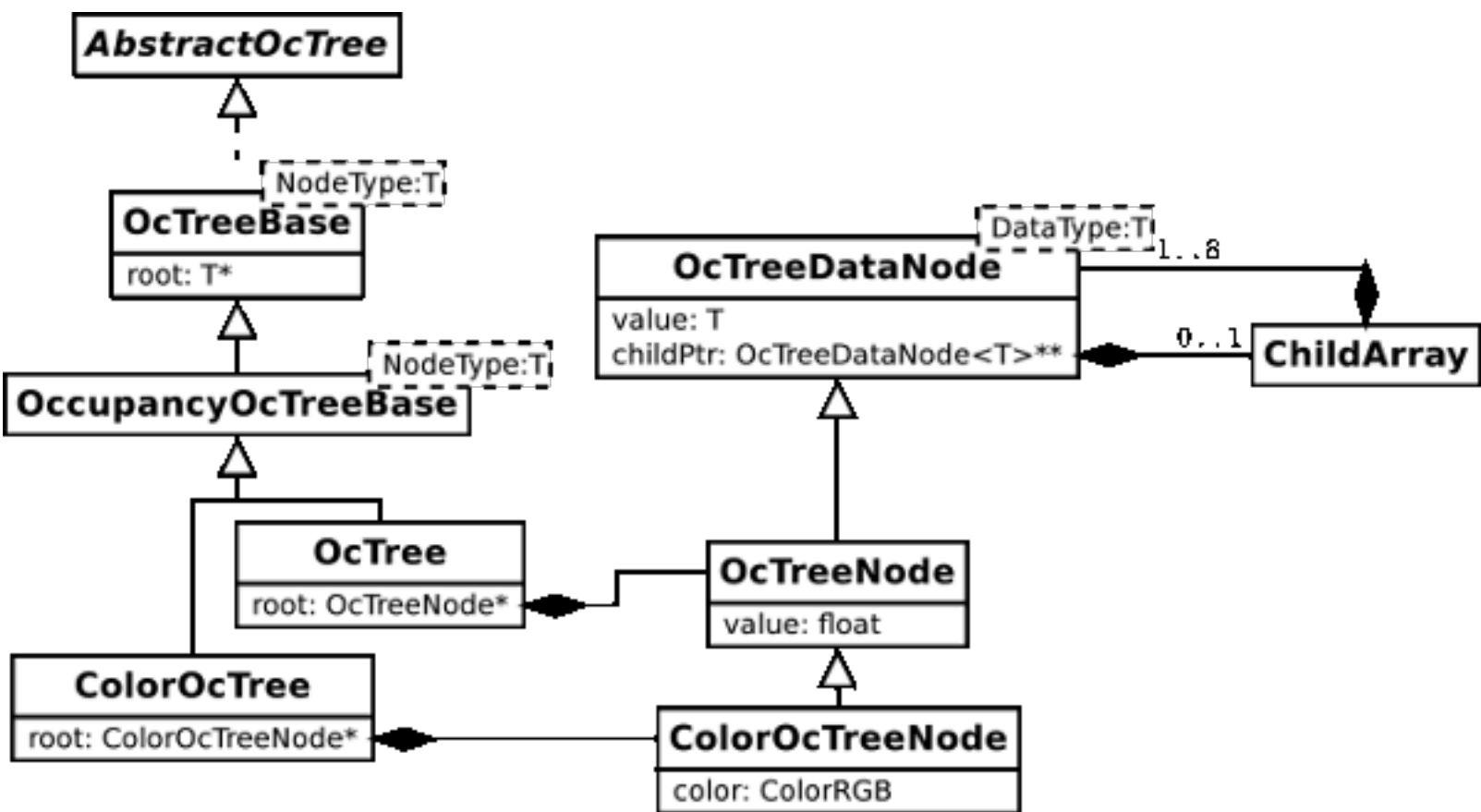
随着我的课题的深入，我逐渐加深了对octomap的内部的细节的理解，也逐步感受到octomap的细节的精妙。我选出一些细节内容与其代码实现，来细讲一些目前网上没有的内容。

相关链接

- 论文：Hornung, Armin, et al. “OctoMap: An efficient probabilistic 3D mapping framework based on octrees.” Autonomous robots 34.3 (2013): 189-206.
- [octomap github](#)

内容

类间关系



- 空心箭头：表示继承关系。
- 实心棱形：表示模板参数

这些class的内容为

- AbstractOcTree: 万物起始。在这个类中定义了一些八叉树应该有的纯虚方法，包括但不限于：
 - 设置与读取八叉树分辨率。
 - 获取八叉树的type string
 - prune/expand/clear 等等方法。
- OcTreeBase: 这个类在继承了AbstractOcTree的基础上，完整实现了octree的数据结构，包括八叉树的search\光线投射\更新node等等。
- OccupancyOcTreeBase: 在继承了OcTreeBase的基础上，引入了logOdds来表示node的占据概率。新增了insertPointcloud方法来一次性插入点云。
- OcTree/ColorOcTree: 继承OccupancyOcTreeBase，是OcTree的具体实现，基本上没有新加什么东西，ColorOcTree给Node增加了颜色方法。

数据结构

数据结构其实octomap的文章粗略说过了，所谓八叉树即在每一层的基础上，对每层的node八等分，作为下一层的node。

八叉树一般是16层,即最上层(1层)的voxel的边长是最下层(16)层的256倍。假设八叉树的分辨率（即最下层的voxel的边长）为0.1m，则该八叉树最多能表示边长为25.6m的立方体区域，超过该区域则无法表示。但是，可以通过设置八叉树的最大深度为32来将边长扩展到6553.6m，这对大部分SLAM问题都算是满足条件的了。

八叉树地图的最底层的栅格总数是固定的，如果八叉树的最大深度为16，则最底层最多会有65536个voxel。然而，八叉树并非初始化时，便给最底层以及其它层的voxel分配好了内存空间。只有当某个区域有点云存在时，八叉树才会从最上层开始一步步分裂，直到这些点被分裂出来的最底层的voxel所包围。详见 OccupancyOcTreeBase.hxx 文件里的updateNode() 以及 updateNodeRecurs()函数。

prune 剪枝

八叉树是存在剪枝的。是自底向上的剪枝，从最底层 16层开始，判断上一层的节点的八个children是否都存在，并且logOdd的值是一样的，如果一样的话，则这八个children都会被剪枝，只保留parent node。详见prune()/pruneRecurs()/pruneNode()三个函数。

迭代器

octree里有两个迭代器，octree->begin(uint depth)与octree->begin_leafs()。实际上begin_leafs()就是octree->begin(16)。作为一个特殊的数据结构，其迭代器实际上是采用广度优先搜索的方式，将某一个特定深度的所有的node全部得到后放到一个vector容器里，然后再迭代这个容器里的node。详见OcTreeIterator.hxx

OcTreeKey & octree->search

OcTreeKey 是每一个Node的类似索引的数据类型，由三个unsigned int构成，分别表示了三个维度方向上的索引。每个Node都有唯一的OcTreeKey，但是一个OcTreeKey可以对应多个深度上不同的voxel,只要这些voxel的中心点一致。

因为一个OcTreeKey可以对应多个深度上的不同的voxel，因此在search的时候，需要指定search的深度。Search算法是一个自顶而下的算法。先从root节点开始搜索，向着底层搜索，并且用getNodeChild()函数来计算所需要的查找的Node在当前Node的哪个Child中，然后继续沿着这个child搜索。

从search的原理中可以看出，search是单向的，可以从某个OcTreeKey或者某个坐标搜索到某个node，但是给某个node的指针，没有直接的方法能够获得这个node的深度、三维坐标以及OcTreeKey。

本文遵守 **Attribution-NonCommercial 4.0 International** 许可协议。



上篇

c++的反射

下篇

机器人常用的三维地图表达形式